

Day 8 lab: object detection on the Raspberry Pi

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

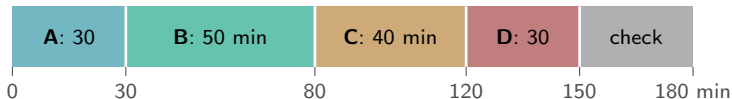
- ▶ **Goal:** your group has a live detection on the Raspberry Pi 5, with a custom model for two classes: a bottle and a cup.
- ▶ **Board:** the Raspberry Pi 5 with the camera.
- ▶ **Deliverable:** a live demonstration, and the frame-rate table and the Decision Log in the file `report.md`.

From the lecture to the lab

The lecture gave the method: the candidates of a detector, the suppression, the IoU, the mAP, and the four decisions of a deployment. All its times are from an x86 processor. In the lab, you train a detector, you export it, and you measure its frame rate and its accuracy for the board.

Key point: A frame rate has no value without the accuracy of the same file. The table of this lab has the two numbers in each row.

Plan of the three hours



Part	Work	Time
A	Pre-trained models: an SSD model and a YOLO model on test images, on the board	30 min
B	Custom model: train a YOLO model on a small dataset with two classes, on the laptop	50 min
C	Export and deploy: NCNN and LiteRT int8, and the live detection with the camera	40 min
D	Measure: the frame rate for two image sizes and two precisions, and the accuracy loss of int8	30 min
Check by the instructor		30 min

Write each result in `report.md` when you get it.

Hardware for each group

- ▶ Raspberry Pi 5 (8 GB) with the active cooler and the card of your group: pi-NN.
- ▶ Power supply, 27 W.
- ▶ Camera Module 3 with its cable.
- ▶ Laptop in the Wi-Fi edgeai-lab.
- ▶ A bottle and a cup.

Software

- ▶ On the board: the environment `~/yolo` with `ultra`lytics 8.4.171, NCNN, and LiteRT.
- ▶ On the laptop: the new environment `~/day08_env` with the file `requirements.txt`: the same version of the package. It needs 2.7 GB.
- ▶ A browser on the laptop, for the live image.

Lab files

Labs/day08/ on the laptop, `~/edgeai/day08/` on the board

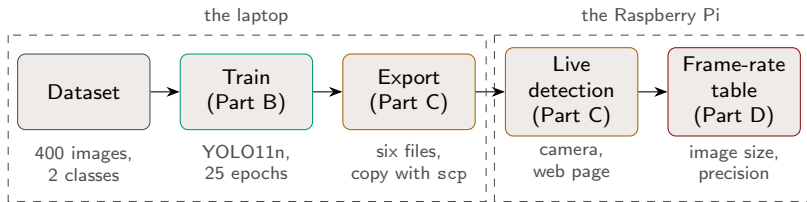
Each new terminal on the board

`cd ~/edgeai/day08` and `source ~/yolo/bin/activate`

The camera cable

Do not connect or remove it when the power is on.

Two computers, one detector



- ▶ Part A is not in the diagram: it runs two pre-trained models on the board. It needs no file of the laptop.
- ▶ The training needs PyTorch, and the export to LiteRT does not run on the board. The two steps stay on the laptop.
- ▶ The download of the dataset and the training run in the background. One student of the group can start sections 0 to 2 of the notebook during Part A.

Key point: The board gets no PyTorch file of your model. It gets the exported files, and each file has one fixed image size.

Step	Work	Time
1	Start the board, copy two folders, and connect	8 min
2	Get two test images: bus.jpg, and a photo of your desk	4 min
3	The SSD model, with Task A1	9 min
4	The YOLO model: two image sizes, two IoU thresholds	9 min

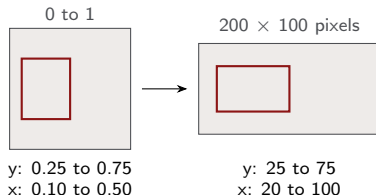
```
ssh edge@pi-NN.local "mkdir -p ~/edgeai/day08/images"    # on the laptop
scp -r day08/pi day08/models edge@pi-NN.local:~/edgeai/day08/
ssh edge@pi-NN.local                                     # then on the board
cd ~/edgeai/day08
source ~/yolo/bin/activate
rpicam-jpeg --output images/desk.jpg --width 1280 --height 960
```

- ▶ Run the first two commands in the folder Labs/. NN is the number of your group.
- ▶ bus.jpg is the image of the lecture. Step 2 of the README gives the command that copies it from the package.
- ▶ Put the bottle and the cup in front of the camera for the photo.

`python pi/detect_ssd.py images/bus.jpg` runs SSD MobileNet V1 of the kit lab:

```
Input:  shape [1, 300, 300, 3], type uint8
Output 0: shape [1, 10, 4], type float32
First inference: ... ms. Median of the next 20 runs: ... ms.
Rows in the output: 10. Rows with a score of 0.50 or more: ...
Task A1: not complete
```

- ▶ **Task A1:** the function `to_pixel_boxes`. A box of the model is `ymin, xmin, ymax, xmax` from 0 to 1. Change it to pixels of the image. Keep the boxes above the score threshold.
- ▶ The diagram is the check of the script.
- ▶ **You record:** the four outputs, the times, and the detections.



```
python pi/detect_yolo.py images/bus.jpg           # YOLO11n, 640 pixels
python pi/detect_yolo.py images/bus.jpg --imgsz 320
python pi/detect_yolo.py images/bus.jpg --iou 0.9
python pi/detect_yolo.py images/desk.jpg --output yolo_desk.jpg
```

Thresholds: score 0.25, IoU 0.70. Image size: 640.

Load: ... ms. First run: ... ms.

Median of the next 5 runs, in ms:

pre-processing ... inference ... post-processing ... complete call ...

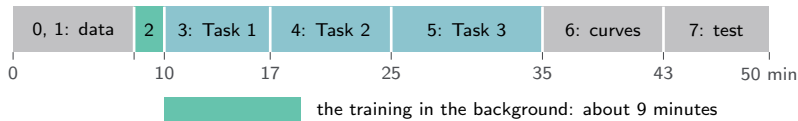
Detections: ...

Prediction, before the second and the third command:

- ▶ The image size goes from 640 to 320 pixels. By which factor does the inference time change?
 - ▶ The IoU threshold goes from 0.7 to 0.9. Do you get more detections, or fewer?
-
- ▶ The first run downloads the weights `yolo11n.pt` (5.4 MB).
 - ▶ **You record:** the time of each step, the detections, and the result for the photo of your desk with each model.

Part B: custom model (50 min)

Work on the laptop, in the notebook `custom_detector.ipynb`. The bar shows its sections.

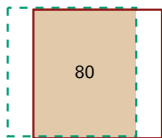


Split	Images	bottle	cup	Use
train	240	172	185	The training changes the weights.
valid	60	41	43	Selects the best epoch. Gives the calibration images for int8.
test	100	71	77	The numbers of your report.

- ▶ Section 1 downloads the 400 images (about 60 MB). Section 2 starts the training. Continue with Task 1 while it runs.
- ▶ The cell after a task prints Task N: complete or not complete.
- ▶ **You record** (section 1): does each box touch its object on the four sides? Which object has no label?

Source: Images and labels: COCO 2017, cocodataset.org.

Task	Function	You write
1	<code>box_iou</code>	The IoU of two boxes: the intersection, the union, and their ratio
2	<code>nms</code>	The suppression for one class: keep the box with the highest score, remove each box with an IoU above the threshold, and do this again
3	<code>count_image</code>	TP, FP, and FN for one class in one image: each detection takes the free label with the highest IoU



$$\text{IoU} = 80/120 = 0.67$$

The checks:

- ▶ Task 1: the two box pairs of the lecture, and 200 random pairs. The diagram shows pair 1.
- ▶ Task 2: `bus.jpg` has 47 candidates above the score 0.25, and 5 detections for the IoU threshold 0.7, as in the lecture.
- ▶ Task 3: five small cases.

You record: the IoU of three pairs, and the number of detections for the two IoU thresholds.

Section	Work
5	The baseline: the pre-trained YOLO11n on the 100 test images, at 320 and at 640 pixels. Your function counts TP, FP, and FN.
6	The result of the training: one line for each epoch, then the curves of the two losses and of the mAP
7	Your model on the 100 test images, and the comparison with the baseline. The validation images selected the best epoch, so they are not a fair test.

Two predictions. Write each one in the report before you run the cell:

- ▶ Section 5: is the recall at 320 pixels lower than at 640 pixels?
 - ▶ Section 7: your model has 2 classes and saw 240 images. The pre-trained model has 80 classes and saw 118 000 images. Which model has the higher recall for the two classes?
- ▶ **You record:** the best epoch with its mAP, the tables of the three models, and one false positive and one false negative of the figure.

The baseline

pre-trained YOLO11n
80 classes, 118 000 images

Your model

the same network, a new head
2 classes, 240 images

- ▶ In the example, the baseline had the higher recall for the two classes. This is a normal result: 240 images do not pass a model that learned the two classes from 118 000 images.
- ▶ A custom model is the correct tool in two cases:
 - The pre-trained model does not know your classes. Example: the box and the wheel of the kit lab.
 - Your images are different from the images of COCO: a different camera position, or a different light.
- ▶ With more images of your own camera, the custom model becomes better than the baseline.

Key point: Measure a baseline before you train. Then you know what the training gave.

Part C: export and deploy (40 min)

Step	Work	Time
1	Export: sections 8 to 10 of the notebook, on the laptop	15 min
2	Copy the six files to the board	3 min
3	Check your model with the photo of your desk	5 min
4	Live detection with the camera	12 min
5	Demonstration for the instructor	5 min

```
python pi/detect_yolo.py images/desk.jpg \  
    --model models/cupbottle_320_ncnn_model --imgsz 320  
python pi/live_detect.py --model models/cupbottle_320_ncnn_model  
python pi/live_detect.py --model models/cupbottle_320_int8.tflite  
python pi/live_detect.py --model models/cupbottle_320_ncnn_model --conf 0.5
```

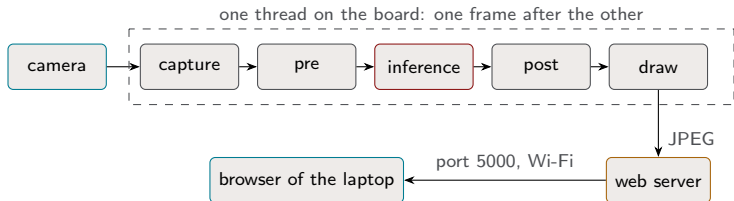
- ▶ Step 2: use the scp command of section 10 of the notebook.
- ▶ Step 3: give the image size of the file with `--imgsz`. Compare the result with the pre-trained model of Part A.
- ▶ Step 4: stop the application with Ctrl+C before you start it with a different file.



- ▶ A name gives the three properties: `cupbottle_320_int8.tflite`.
- ▶ Section 9 measures the mAP of each LiteRT file. An NCNN folder has the same `float32` weights: use the value of the LiteRT `float32` file.
- ▶ The test set has 100 images. A difference of 0.02 of the mAP is not a clear result.

Three predictions, before the cells of sections 8 and 9:

- ▶ How large is the `int8` file, as a part of the `float32` file?
- ▶ How many points of mAP50-95 does the `int8` file lose?
- ▶ Your model learned at 320 pixels. Is its mAP at 640 pixels higher, the same, or lower?



```
Model: cupbottle_320_ncnn_model, NCNN, float32, image size 320
Open http://<name of this board>:5000 in a browser. Stop with Ctrl+C.
... frames/s | capture ... pre ... inference ... post ... draw ... ms | 1 bottle, 1 cup | ... C
```

- ▶ Open `http://pi-NN.local:5000` on the laptop. You see the live image with the boxes, the frame rate, and the time of each step.
- ▶ Put the bottle and the cup in front of the camera. Move them, change the distance, and change the background.
- ▶ Then start the `int8` file, and the first file with the score threshold 0.5.
- ▶ **You record:** the frame rate and the inference time of each command, the objects that the model finds, the objects that it misses, and the wrong detections.

Part D: measure (30 min)

Step	Work	Time
1	Predict: write your answer for each question of the report	5 min
2	Task D1: the function <code>frame_rate</code> in <code>pi/bench_detect.py</code>	5 min
3	Measure: run the script two times	10 min
4	Fill the frame-rate table, and calculate the ratios	5 min
5	Compare with your predictions, and write the Decision Log	5 min

model	runtime	type	size	capture	pre	inference	post	frames/s	objects
cupbottle_320_ncnn_model	NCNN	float32	320		...	...	...		...
...									
cupbottle_640_int8.tflite	LiteRT	int8	640		...	...	...		...

- ▶ `python pi/bench_detect.py` measures the six files: the median time of each step for 30 frames of the camera. It also writes `results.csv`.
- ▶ Task D1: one frame needs the sum of the times of the four steps. The check is the example of the lecture: 325 ms for one frame give 3.1 frames in each second.

Prediction, before you run the script:

- ▶ `float32`: which runtime is faster, NCNN or LiteRT?
 - ▶ The gain of the frame rate from 640 to 320 pixels: 4, more than 4, or less than 4?
 - ▶ LiteRT: is `int8` faster than `float32`? By which factor?
 - ▶ Which step is the largest after the inference?
-
- ▶ Stop the live application. Point the camera at the bottle and the cup. Use the second run for the table: the difference between the two runs is the noise of your measurement.
 - ▶ **You record**: the six rows with the mAP50-95 of section 9, the two temperatures, and the ratios: 320 against 640 pixels, `int8` against `float32`, NCNN against LiteRT.
 - ▶ The lecture measured on an x86 processor: `int8` was slower than `float32`, and NCNN was slower than LiteRT. What do you measure?

The instructor checks this at the end of the lab:

- ▶ ✓ `python pi/detect_ssd.py images/bus.jpg` prints Task A1: complete and a list of detections.
- ▶ ✓ The notebook prints Tasks complete: 1, 2, 3 in its last cell.
- ▶ ✓ The live image shows a box for the bottle and a box for the cup, with the custom model.
- ▶ ✓ `python pi/bench_detect.py` prints Task D1: complete.
- ▶ ✓ The frame-rate table of the report has four measured rows or more: two image sizes and two precisions.
- ▶ ✓ The report gives the accuracy loss of the `int8` file.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

Hand in: the file `report.md`. Show the live image to the instructor.

Write about 100 words. State one design decision, give your measured numbers, and name the trade-off.

A product with a Raspberry Pi 5 and a camera must count the bottles and the cups on a table, with 10 frames in each second or more.

- ▶ Which file do you select: which runtime, which image size, and which precision?

A good Decision Log:

- ▶ gives the frame rate and the mAP50-95 of your selection and of one other file from your table,
- ▶ compares the frame rate with the target of 10 frames in each second,
- ▶ names what you lose with your decision. Example: the file for 320 pixels is faster, and it finds fewer small objects.

If a part does not work, and common problems

Problem	Fix or fallback
the package ultralytics is not available	The environment is not active: <code>source ~/yolo/bin/activate</code>
The camera gives no photo	Use <code>images/bus.jpg</code> in Part A, and <code>--source images/</code> in Parts C and D. Tell the instructor.
The download of the dataset stops	Run <code>python get_dataset.py</code> again. It keeps the complete images.
The training is not complete in time	The instructor gives the file <code>cupbottle.pt</code> . Put it into <code>models/</code> and run the notebook from section 3.
A task is not complete in time	Use the cell or the file of the folder <code>solutions/</code> . Write this in the report.
The export to LiteRT stops with an error	Do Parts C and D with the two NCNN folders. The instructor gives the four LiteRT files.
The live image has many wrong boxes	Use the image size of the file name: <code>cupbottle_320...</code> runs with 320.
The custom model does not find your objects	Use <code>--conf 0.15</code> , a plain background, and a distance of about 50 cm.

The README of the lab has the complete list.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.